Eğitim



TypeScript'e giriş

JavaScript'e tip güvenliği kazandırmak: temelden uygulamaya

Plan

Bugünün yol haritası

Bölüm	Konu
1	Neden TypeScript? ve kurulum
2	Temel tipler
3	Fonksiyonlar ve nesneler
4	Union, literal tipler ve narrowing
5	Generics
6	Utility types ve as const
7	Uygulama: tipli API çağrısı
8	Kapanış ve sonraki adımlar

Bölüm 1

Neden TypeScript?

Fark, çalışma modeli ve kurulum

Hatayı çalıştırmadan yakalamak

JavaScript: hata çalışırken çıkar

```
function kdvEkle(fiyat) {
  return fiyat * 1.2;
}

kdvEkle("100 TL");
```

```
Çıktı: NaN. Hata yok, yanlış sonuç var.
```

TypeScript: hata yazarken çıkar

```
function kdvEkle(fiyat: number) {
  return fiyat * 1.2;
}

kdvEkle("100 TL");
```

```
Argument of type 'string' is not assignable to
parameter of type 'number'.
```

TypeScript = JavaScript + tipler



Tipler silinir

Çalışma zamanında tip yoktur; çıktı düz JavaScript'tir.

Her JS geçerli TS

Mevcut projeyi adım adım TypeScript'e geçirebilirsiniz.

Editör desteği

Otomatik tamamlama, yeniden adlandırma ve anında hata gösterimi.

Kurulum: üç komut yeter

Terminal

```
$ npm init -y
$ npm install -D typescript tsx
$ npx tsc --init
```

tsconfig.json (önemli kısım)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "strict": true,
    "outDir": "dist"
  }
}
```

Çalıştırmak için npx tsx index.ts, derlemek için npx tsc. Kurulumsuz alternatif: typescriptlang.org/play

Bölüm 2

Temel tipler

Tip yazımı, inference, any ve unknown, null kontrolü

Temel tipler ve tip belirtme

```
const ad: string = "Ayşe";
const yas: number = 21;
const aktif: boolean = true;

let puan: number = 90;
puan = "doksan";
```

```
Type 'string' is not assignable to
type 'number'.
```

Sözdizimi

`değişken: tip = değer`

Tip adları küçük harfle yazılır: string, number, boolean. Büyük harfli String ve Number kullanılmaz.

Type inference: her şeyi yazmak gerekmez

```
let sehir = "Ankara"; // string
let adet = 3; // number

sehir = 42;
```

```
Type 'number' is not assignable to
type 'string'.
```

Ne zaman yazmalı?

- Başlangıç değeri varsa TypeScript tipi kendisi çıkarır.
- Fonksiyon parametrelerinde tipi siz yazarsınız.
- Boş başlangıçta (let x;) tipi siz belirtirsiniz.

any ve unknown: iki farklı kapı

any: kontrolü kapatır

```
let veri: any = "merhaba";
veri.olmayanMetot();
```

```
Derleme: hata yok. Çalışma: TypeError.
```

unknown: önce kontrol ister

```
let veri: unknown = "merhaba";
veri.toUpperCase();
if (typeof veri === "string") {
  veri.toUpperCase();
}
```

```
'veri' is of type 'unknown'.
```

Array ve tuple

Array: aynı tipten liste

```ts
const notlar: number[] = [80, 95];
const isimler: Array<string> = ["Ali"];

notlar.push("100");
```

```
Argument of type 'string' is not assignable to
parameter of type 'number'.
```

Tuple: sabit uzunluk ve sıra

```ts
const kisi: [string, number] =
  ["Ayşe", 21];

kisi[0]; // string
kisi[1]; // number
```

null ve undefined: strict modun faydası

Hata: null olabilir

```
function selamla(ad: string | null) {
  return ad.toUpperCase();
}
```

```
'ad' is possibly 'null'.
```

Doğru: güvenli erişim

```
function selamla(ad: string | null) {
  return ad?.toUpperCase() ?? "Misafir";
}
```

```
?. null ise durur, ?? yedek değer verir.
```

string | null bir union tipidir; ayrıntısını birazdan göreceğiz.

Alıştırma 1

Hataları bulun ve düzeltin

Aşağıdaki kodda 3 hata var. Editörde kırmızı alt çizgiyi takip edin.

```
let urun: string = "Defter";
let adet: number = "5";
let fiyatlar: number[] = [10, 20, "30"];
let stokta: boolean = 1;
```

Çözüm için tipi değil, değeri düzeltin.

Bölüm 3

Fonksiyonlar ve nesneler

Parametre ve dönüş tipleri, interface, type

Fonksiyon tipleri

```
function topla(a: number, b: number): number {
  return a + b;
}

const carp = (a: number, b: number) ⇒ a * b;
```

Opsiyonel

`unvan?: string`

Varsayılan değer

`adet: number = 1`

Dönüş yoksa

void değer döndürmez, never hiç dönmez (hata fırlatır).

Nesne tipleri: interface

Şekli tanımla

```ts
interface Ogrenci {
  id: number;
  ad: string;
  mail?: string;
  readonly kayitYili: number;
}
```

Kullan

```ts
const o: Ogrenci = {
  id: 1,
  ad: "Zeynep",
  kayitYili: 2025,
};

o.kayitYili = 2026;
```

```
Cannot assign to 'kayitYili' because it is a
read-only property.
```

interface mı, type mı?

Özellik	interface	type
Nesne şekli tanımlama	Evet	Evet
Genişletme	extends	& ile birleştirme
Union (A | B)	Hayır	Evet
Aynı isimle ekleme	Evet (declaration merging)	Hayır
Tuple ve takma ad	Hayır	Evet

Basit kural: nesneler için interface, geri kalan her şey için type.

Alıştırma 2

Kendi tipinizi tanımlayın

Urun adında bir interface yazın: id, ad, fiyat ve opsiyonel aciklama. Sonra indirimliFiyat(urun, oran) fonksiyonunu tipleriyle yazın.

```
// 1) interface Urun { ... }
// 2) function indirimliFiyat(urun, oran) { ... }

const kalem: Urun = { id: 1, ad: "Kalem", fiyat: 50 };
console.log(indirimliFiyat(kalem, 0.2)); // 40
```

Bölüm 4

Union, literal tipler ve narrowing

Bu eğitimin en önemli bölümü

Union: “şu veya bu”

Birden fazla tipten biri

```ts
type Id = string | number;

let kullaniciId: Id = 101;
kullaniciId = "A-101";
kullaniciId = true;
```

```
Type 'boolean' is not assignable to type 'Id'.
```

Yalnızca ortak üyeler kullanılabilir

```ts
function yazdir(deger: string | number) {
  console.log(deger.toFixed(2));
}
```

```
Property 'toFixed' does not exist on type
'string | number'.
```

Literal tipler: izin verilen değerleri sınırla

```ts
type Rol = "admin" | "ogretmen" | "ogrenci";

let rol: Rol = "admin";
rol = "mudur";
```

```
Type '"mudur"' is not assignable to type 'Rol'.
```

Neden işe yarar?

- Yazım hatalarını derleme anında yakalar.
- Editör geçerli değerleri otomatik önerir.
- enum'a hafif bir alternatiftir.

Narrowing: tipi kod akışıyla daraltmak

typeof

```ts
function yazdir(d: string | number) {
  if (typeof d === "string") {
    return d.toUpperCase();
  }
  return d.toFixed(2);
}
```

in operatörü

```ts
type Kus = { uc: () => void };
type Balik = { yuz: () => void };

function hareket(h: Kus | Balik) {
  if ("uc" in h) h.uc();
  else h.yuz();
}
```

Aynı mantık: typeof, in, instanceof ve === null kontrolü.

Discriminated union: en güçlü kalıp

```ts
type Sonuc =
  | { durum: "basarili"; veri: string }
  | { durum: "hata"; mesaj: string };

function goster(s: Sonuc) {
  if (s.durum === "basarili") {
    console.log(s.veri);
  } else {
    console.log(s.mesaj);
  }
}
```

Ortak alan = etiket

durum alanına bakınca TypeScript hangi şekille çalıştığınızı bilir.

else dalında s.mesaj güvenle kullanılır.

API cevapları ve arayüz durumlarında çok sık kullanılır.

Alıştırma 3

Union ve narrowing

Sipariş durumu için “bekliyor”, “yolda” ve “teslim” değerlerinden oluşan bir tip tanımlayın. durumYazisi her duruma farklı bir Türkçe metin döndürsün.

```
type Durum = /* ??? */;

function durumYazisi(d: Durum): string {
  // if veya switch ile her duruma bir metin döndürün
}

durumYazisi("yolda");
durumYazisi("kayip"); // hata vermeli
```

Bölüm 5

Generics

Tipi bir parametre gibi dışarıdan almak

Problem: her tip için ayrı fonksiyon mu?

Kopyala-yapıştır

```
function ilkSayi(l: number[]) {
  return l[0];
}
function ilkYazi(l: string[]) {
  return l[0];
}
```

any ile tip kaybı

```
function ilk(l: any[]) {
  return l[0];
}

const x = ilk([1, 2]);
x.olmayanMetot(); // hata yok
```

```
x tipi any oldu, hata yakalanmadı.
```

Çözüm: tipi fonksiyona bir parametre gibi vermek.

Generic fonksiyon: <T>

```ts
function ilk<T>(liste: T[]): T {
  return liste[0];
}

const a = ilk([1, 2, 3]);
const b = ilk(["x", "y"]);
```

T bir tip değişkenidir

Fonksiyon çağrıldığı yerde TypeScript T'yi kendisi bulur.

```
a: number
```

```
b: string
```

Her gün kullandığınız generic'ler

```ts
const sayilar: Array<number> = [1, 2, 3];
const sozVer: Promise<string> = Promise.resolve("hazır");

const harita = new Map<string, number>();
harita.set("Ali", 90);
harita.set("Veli", "100");
```

```
Argument of type 'string' is not assignable to parameter of type 'number'.
```

Promise<T>, async fonksiyonların dönüş tipidir. Uygulamada kullanacağız.

Kısıt koymak: extends

```ts
function uzunluk<T extends { length: number }>(x: T) {
  return x.length;
}

uzunluk("merhaba");
uzunluk([1, 2, 3]);
uzunluk(42);
```

```
Argument of type 'number' is not assignable to parameter of type '{ length: number; }'.
```

T extends X: T en azından X'in şekline sahip olmalı.

Alıştırma 4

Kendi generic fonksiyonunuz

Bir dizinin son elemanını döndüren sonEleman fonksiyonunu generic olarak yazın.

```
function sonEleman(/* ? */) {
  // ?
}

const s1 = sonEleman([10, 20, 30]); // number
const s2 = sonEleman(["a", "b"]); // string
```

Bölüm 6

Utility types ve as const

Hazır tip araçları ve değerden tip türetmek

Utility types: hazır tip araçları

Temel tip: interface Kullanici { id: number; ad: string; mail: string }

Araç	Ne yapar?	Örnek
Partial<T>	Tüm alanları opsiyonel yapar	Partial<Kullanici>
Required<T>	Tüm alanları zorunlu yapar	Required<Kullanici>
Pick<T, K>	Seçilen alanları alır	Pick<Kullanici, "id" | "ad">
Omit<T, K>	Seçilen alanları çıkarır	Omit<Kullanici, "mail">
Record<K, V>	Anahtar-değer sözlüğü	Record<string, number>
ReturnType<F>	Fonksiyonun dönüş tipi	ReturnType<typeof ilk>

enum yerine union ve as const

enum: ekstra JavaScript kodu üretir

```
enum Rol {
  Admin,
  Ogrenci,
}
```

Önerilen: as const

```
const ROLLER = ["admin", "ogr"] as const;
type Rol = (typeof ROLLER)[number];
// "admin" | "ogr"
```

as const, diziyi salt okunur ve literal yapar; tipi değerlerden türetirsiniz. Her iki yolu da kodda göreceksiniz.

Bölüm 7

Uygulama: tipli API çağrısı

Öğrendiklerimizi birleştiriyoruz: canlı kodlama

Görev: kullanıcı listesini tipli çekmek

1. Kullanici interface'ini tanımlayın (id, name, email).
2. Sonuc<T> discriminated union'ını yazın.
3. async fonksiyondan Promise<Sonuc<Kullanici[]>> döndürün.
4. fetch ile veriyi çekin, res.ok kontrolü yapın.
5. Başarılıysa kullanıcı adlarını yazdırın.

Veri kaynağı

jsonplaceholder.typicode.com/users

Adım 1: tipleri tanımla

Veri şekli

```ts
interface Kullanici {
  id: number;
  name: string;
  email: string;
}
```

Sonuç şekli (generic)

```ts
type Sonuc<T> =
  | { durum: "basarili"; veri: T }
  | { durum: "hata"; mesaj: string };
```

Sonuc<T> her veri tipiyle çalışır: Sonuc<Kullanici[]>, Sonuc<string>…

Adım 2: tipli fetch fonksiyonu

```
async function kullanicilariGetir(): Promise<Sonuc<Kullanici[]>> {
  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  if (!res.ok) return { durum: "hata", mesaj: "Sunucu hatası" };
  const veri: Kullanici[] = await res.json();
  return { durum: "basarili", veri };
}

const sonuc = await kullanicilariGetir();
if (sonuc.durum === "basarili") sonuc.veri.forEach(k => console.log(k.name));
```

Dikkat: res.json() any döner. Gerçek projede Zod gibi bir kütüphaneyle doğrulayın.

Kapanış

Bugün neler öğrendik?

Özet

- Temel tipler ve inference
- Fonksiyon ve nesne tipleri
- Union, literal tipler, narrowing
- Generics ve utility types
- Tipli API çağrısı

Sonraki adımlar

- strict: true her zaman açık olsun
- any yerine unknown kullanın
- React + TypeScript, Node + TypeScript
- Zod ile çalışma zamanı doğrulaması